VOL. 03

JavaScript

The language, the browser, and how it really works

Source	javascript.info
Generated	2026-10-10
Level	Beginner to intermediate
Estimated study time	40 hours
Language	English
Volume	Vol. 03 - JavaScript

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 8

Chapter map 8

Chapter 1 - Introduction and running JavaScript 9

1.1 Summary 9

1.2 Concepts 9

Where JavaScript lives 9

Including a script 9

The console 9

1.3 Examples 10

1.4 Common mistakes and best practices 11

1.5 Practice exercises 11

Solutions 11

1.6 Mini test 12

Answer key and explanations 12

Chapter 2 - Values, variables and operators 12

2.1 Summary 12

2.2 Concepts 13

The types 13

Declaring variables 13

Operators 13

Coercion traps 13

2.3 Examples 14

2.4 Common mistakes and best practices 15

2.5 Practice exercises 16

Solutions 16

2.6 Mini test 16

Answer key and explanations 16

Chapter 3 - Control flow and functions 17

3.1 Summary 17

3.2 Concepts 17

Branching 17

Loops 17

Function forms . . . 17
Parameters . . . 18
3.3 Examples . . . 18
3.4 Common mistakes and best practices . . . 20
3.5 Practice exercises . . . 20
Solutions . . . 20
3.6 Mini test . . . 21
Answer key and explanations . . . 21
Chapter 4 - Objects, prototypes and classes . . . 21
4.1 Summary . . . 21
4.2 Concepts . . . 22
Creating and reading . . . 22
`this` . . . 22
Prototype chain . . . 22
Classes . . . 22
Object utilities . . . 23
4.3 Examples . . . 23
4.4 Common mistakes and best practices . . . 24
4.5 Practice exercises . . . 25
Solutions . . . 25
4.6 Mini test . . . 25
Answer key and explanations . . . 26
Chapter 5 - Strings, numbers and arrays . . . 26
5.1 Summary . . . 26
5.2 Concepts . . . 26
String methods . . . 26
Number methods . . . 27
Array methods that matter . . . 27
5.3 Examples . . . 27
5.4 Common mistakes and best practices . . . 29
5.5 Practice exercises . . . 30
Solutions . . . 30
5.6 Mini test . . . 30
Answer key and explanations . . . 30
Chapter 6 - Iterables, Map, Set, destructuring, JSON . . . 31

6.1 Summary . . . 31
6.2 Concepts . . . 31
Map . . . 31
Set . . . 31
Iterables . . . 31
Destructuring . . . 31
JSON . . . 32
6.3 Examples . . . 32
6.4 Common mistakes and best practices . . . 34
6.5 Practice exercises . . . 34
Solutions . . . 34
6.6 Mini test . . . 35
Answer key and explanations . . . 35
Chapter 7 - Scope, closures and advanced functions . . . 35
7.1 Summary . . . 35
7.2 Concepts . . . 36
Scope in one example . . . 36
Closure . . . 36
`this` . . . 36
Currying and composition . . . 37
7.3 Examples . . . 37
7.4 Common mistakes and best practices . . . 39
7.5 Practice exercises . . . 39
Solutions . . . 39
7.6 Mini test . . . 40
Answer key and explanations . . . 40
Chapter 8 - Error handling . . . 40
8.1 Summary . . . 40
8.2 Concepts . . . 40
The syntax . . . 41
Throwing . . . 41
Built-in error types . . . 41
Strict mode turns mistakes into errors . . . 41
8.3 Examples . . . 41
8.4 Common mistakes and best practices . . . 43

8.5 Practice exercises . . . 44
Solutions . . . 44
8.6 Mini test . . . 44
Answer key and explanations . . . 45
Chapter 9 - Asynchronous JavaScript . . . 45
9.1 Summary . . . 45
9.2 Concepts . . . 45
The event loop . . . 45
Promises . . . 45
`async` / `await` . . . 46
9.3 Examples . . . 46
9.4 Common mistakes and best practices . . . 48
9.5 Practice exercises . . . 49
Solutions . . . 49
9.6 Mini test . . . 49
Answer key and explanations . . . 49
Chapter 10 - Modules and code quality . . . 50
10.1 Summary . . . 50
10.2 Concepts . . . 50
Export and import . . . 50
In the browser and in Node . . . 51
A tiny test . . . 51
10.3 Examples . . . 51
10.4 Common mistakes and best practices . . . 52
10.5 Practice exercises . . . 53
Solutions . . . 53
10.6 Mini test . . . 53
Answer key and explanations . . . 54
Chapter 11 - The DOM . . . 54
11.1 Summary . . . 54
11.2 Concepts . . . 54
Selecting . . . 54
Reading and writing . . . 54
Creating and inserting . . . 55
Traversal . . . 55

11.3 Examples . . . 55
11.4 Common mistakes and best practices . . . 56
11.5 Practice exercises . . . 57
Solutions . . . 57
11.6 Mini test . . . 57
Answer key and explanations . . . 57
Chapter 12 - Events and forms . . . 58
12.1 Summary . . . 58
12.2 Concepts . . . 58
Phases . . . 58
The event object . . . 58
Useful events . . . 59
Forms . . . 59
12.3 Examples . . . 59
12.4 Common mistakes and best practices . . . 60
12.5 Practice exercises . . . 61
Solutions . . . 61
12.6 Mini test . . . 61
Answer key and explanations . . . 61
Chapter 13 - Storage, fetch and modern extras . . . 62
13.1 Summary . . . 62
13.2 Concepts . . . 62
Storage . . . 62
`fetch` . . . 62
`URL` . . . 62
13.3 Examples . . . 63
13.4 Common mistakes and best practices . . . 65
13.5 Practice exercises . . . 65
Solutions . . . 65
13.6 Mini test . . . 66
Answer key and explanations . . . 66
Cheat sheet . . . 66
14.1 Running and syntax . . . 66
14.2 Types and checks . . . 67

14.3 Numbers 67
14.4 Strings 67
14.5 Arrays 67
14.6 Objects and classes 67
14.7 Map, Set, JSON 68
14.8 Functions 68
14.9 Scope and closures 68
14.10 Errors 68
14.11 Async 68
14.12 DOM and events 69
14.13 Storage, URL, Intl 69
14.14 The traps to remember 69
Final test 70
15.1 Final test answers 71
Sources 72

How to use this volume

Prerequisites: HTML (Vol. 01). No programming experience is assumed, but you must be comfortable opening a file in an editor and in a browser.

Estimated study time: about 40 hours. This is the largest volume in the library: JavaScript is the behaviour layer of the web, and it is also a complete programming language.

How the chapters work: every chapter has a summary, concepts, runnable examples with their real output, the common mistakes, practice exercises with solutions, and a mini test with an answer key.

The examples in this volume really run. Every `js` block was executed with Node.js and its output compared with the published result; the multi-file module example is executed as two real files. The blocks marked `jsref` are API reference listings, not programs, and the blocks marked `DOM EXAMPLE` need a browser, so those two kinds are the only ones not executed here.

How to run the examples

shell

```shell
node 01-hello.js          # run one file
node --check 01-hello.js  # syntax check only
```

Chapter map

#	Chapter	You will learn
1	Introduction and running JavaScript	where JS runs, the console, the first program
2	Values, variables and operators	types, coercion, `let`/`const`, the operators
3	Control flow and functions	`if`, loops, functions, arrow functions, defaults
4	Objects, prototypes and classes	the object model, `this`, classes
5	Strings, numbers and arrays	the API you use every day
6	Iterables, Map, Set, destructuring, JSON	modern data handling
7	Scope, closures and advanced functions	why closures work, `call`/`apply`/`bind`
8	Error handling	`try`, `catch`, custom errors
9	Asynchronous JavaScript	callbacks, promises, `async`/`await`, the event loop
10	Modules and code quality	ESM, comments, debugging, a test
11	The DOM	selecting, creating and changing elements
12	Events and forms	listeners, delegation, validation
13	Storage, fetch and modern extras	`localStorage`, `fetch`, `Proxy`, `BigInt`, regex

#	Chapter	You will learn
14	Cheat sheet	one-page reference
15	Final test	30 questions with answers

Chapter 1 - Introduction and running JavaScript

1.1 Summary

- JavaScript runs in browsers, on servers (Node), and in many other hosts.
- It is the only language a browser executes natively.
- The console (`console.log`) is your primary debugging tool.
- "use strict" mode is the default in modules and classes.
- One script file, one job: keep behaviour out of the HTML.
- The language is standardised as ECMAScript; new features arrive yearly.
- Every browser ships developer tools with a console and a debugger.
- `typeof` tells you the type of a value at runtime.
- Values change type dynamically: JavaScript is dynamically typed.
- Reading code carefully beats guessing: the console shows the truth.

1.2 Concepts

Where JavaScript lives

output

```
+------------------+-------------------------------------+
| host             | what you can do                     |
+------------------+-------------------------------------+
| browser          | DOM, events, fetch, timers, storage |
| Node.js          | files, network, processes           |
| Deno / Bun       | like Node, with batteries           |
+------------------+-------------------------------------+
```

The language core is the same everywhere; only the surrounding APIs differ.

Including a script

HTML

```
<script src="app.js" defer></script>
```

`defer` downloads the file in parallel and runs it after the HTML is parsed, which is the safe default.

The console

JavaScript

```javascript
console.log("plain");      // a string
console.log({ a: 1 });     // an object
console.table([1, 2, 3]);  // a table
console.error("ops");       // red output
```

1.3 Examples

Example 1: the first program.

JavaScript

```javascript
console.log("Hello, world");
```

output

```
Hello, world
```

Example 2: checking the environment.

JavaScript

```javascript
console.log(typeof window, typeof globalThis, typeof process);
```

output

```
undefined object object
```

`window` only exists in a browser, so `undefined` tells you that you are in Node.

Example 3: several kinds of output.

JavaScript

```javascript
console.log("string", 42, true, null, undefined, { a: 1 }, [1, 2]);
console.info("info uses the same format");
console.warn("warn: careful");
console.error("error: something broke");
```

output

```
string 42 true null undefined { a: 1 } [ 1, 2 ]
info uses the same format
warn: careful
error: something broke
```

console.warn and console.error write to stderr and prefix the line with Warning: and Error: in Node; the values print exactly like console.log.

Example 4: dynamic typing.

JavaScript

```
let value = 1;
console.log(typeof value);
value = "one";
console.log(typeof value);
value = true;
console.log(typeof value);
```

output

```
number
string
boolean
```

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`<script>` in the middle of the body	blocks parsing	`defer` in the head
`alert()` for debugging	blocks the page	`console.log`
No `"use strict"` thinking	silent mistakes	use modules or classes
Mixing many responsibilities in one file	unmaintainable	one concern per file
`var` anywhere	function-scoped surprises	`let` and `const`
Guessing instead of logging	wasted hours	log, then reason

- Open the console before you write any JavaScript.
- Keep the three layers separate: HTML, CSS, JS files.

1.5 Practice exercises

1. Print your name and your favourite number with one `console.log`.
2. How can you tell from JavaScript whether you are in a browser?
3. Which value does `typeof` return for a function?
4. Why is `defer` better than a script at the end of the body?
5. Print an object and a table of two rows.

Solutions

1. `console.log("Ada", 36);`
2. Check `typeof window === "object"` (or `typeof document`).
3. `"function"`.
4. With `defer` the script downloads in parallel and runs after the HTML is parsed, so the page renders sooner and the script cannot hit a missing element.
5. `console.log({ a: 1 });` and `console.table([{ name: "Ada" }, { name: "Alan" }]);`

1.6 Mini test

1. Multiple choice: which method prints to the console? - a) `print()` - b) `console.log()` - c) `echo()` - d) `log()`
2. True or false: JavaScript only runs in browsers.
3. Multiple choice: which script attribute runs the file after parsing? - a) `async` - b) `defer` - c) `later` - d) `type="module"`
4. Short code question: print the type of the value `42`.
5. Multiple choice: what is JavaScript standardised as? - a) HTML - b) ECMAScript - c) JSON - d) WASM
6. True or false: `typeof null` returns `"null"`.
7. Short answer: name two debugging tools a browser gives you.

Answer key and explanations

1. b - `console.log()`.
2. False - Node.js, Deno and Bun run JavaScript on servers.
3. b - `defer`.
4. `console.log(typeof 42);` prints `number`.
5. b - ECMAScript.
6. False - it returns `"object"`, a long-standing quirk.
7. Any two of: the console, the debugger with breakpoints, the Elements panel, the Network panel, the Sources panel.

Chapter 2 - Values, variables and operators

2.1 Summary

- Seven primitive types: `string`, `number`, `boolean`, `null`, `undefined`, `bigint`, `symbol`.
- Objects (including arrays and functions) are everything else.
- `let` is block-scoped and reassignable; `const` is not reassignable.
- `==` coerces, `===` compares type and value: always use `===`.
- `+` adds numbers and concatenates strings; everything else converts to numbers.
- Template literals embed expressions with `${...}`.
- Numbers are IEEE-754 doubles, so `0.1 + 0.2` is not exactly `0.3`.
- `NaN` means "not a number" and is not equal to itself.
- Truthiness: `""`, `0`, `NaN`, `null`, `undefined` and `false` are falsy.
- `??` returns the right side only for `null` or `undefined`.

2.2 Concepts

The types

JavaScript

```javascript
typeof "text"        // "string"
typeof 42            // "number"
typeof 42n           // "bigint"
typeof true          // "boolean"
typeof undefined     // "undefined"
typeof Symbol("x")   // "symbol"
typeof null          // "object"   (quirk)
typeof {}            // "object"
typeof []            // "object"
typeof function () {}// "function"
```

Declaring variables

JavaScript

```javascript
let counter = 0;
counter = 1;          // allowed

const max = 10;
// max = 11;          // TypeError

const list = [1, 2];
list.push(3);         // allowed: const protects the binding, not the value
```

Operators

Kind	Operators
Arithmetic	`+ - * / % **`
Assignment	`= += -= *= /= %=`
Comparison	`== != === !== > >= < <=`
Logical	`&& || ! ??`
Bitwise	`& | ^ ~ << >> >>>`
String	`+` and template literals
Type	`typeof`, `instanceof`, `in`

Coercion traps

JavaScript

```javascript
console.log("5" + 3, "5" - 3);
console.log("5" * "2");
console.log([] + [], [] + {});
console.log(1 < 2 < 3, 3 > 2 > 1);
```

output

```
53 2
10
 [object Object]
true false
```

`+` concatenates when either side is a string; the other operators convert both operands to numbers. Comparisons return booleans, and a boolean is `1` or `0` when it meets an operator, which is why `3 > 2 > 1` is `false`.

2.3 Examples

Example 1: primitives and objects.

JavaScript

```js
const s = "text";
const n = 42;
const o = { a: 1 };
const arr = [1, 2, 3];

console.log(typeof s, typeof n, typeof o, typeof arr);
console.log(Array.isArray(arr), arr instanceof Array);
```

output

```
string number object object
true true
```

Example 2: `===` versus `==`.

JavaScript

```js
console.log(1 === "1", 1 == "1");
console.log(0 === false, 0 == false);
console.log(null === undefined, null == undefined);
```

output

```
false true
false true
false true
```

Example 3: template literals.

JavaScript

```js
const name = "Ada";
const age = 36;
console.log(`${name} is ${age} years old. In ten years: ${age + 10}.`);
console.log(`multi
line`);
```

output

```
Ada is 36 years old. In ten years: 46.
multi
line
```

Example 4: coercion and `NaN`.

JavaScript

```javascript
console.log("5" + 3, "5" - 3);
console.log(Number("abc"), Number("12"));
console.log(NaN === NaN, Number.isNaN(NaN));
console.log(typeof NaN);
```

output

```
53 2
NaN 12
false true
number
```

Example 5: falsy values.

JavaScript

```javascript
const checks = ["", 0, NaN, null, undefined, false];
console.log(checks.map((v) => Boolean(v)).join(","));
console.log(checks.filter(Boolean).length);
```

output

```
false,false,false,false,false,false
0
```

Example 6: logical and nullish.

JavaScript

```javascript
console.log(0 || "fallback");     // || returns on falsy
console.log(0 ?? "fallback");     // ?? only on null/undefined
console.log(null ?? "fallback");
console.log(true && "second");
```

output

```
fallback
0
fallback
second
```

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
== for comparisons	surprises with coercion	===
Comparing floats with ===	precision errors	compare with a tolerance
`var`	function scope, hoisting surprises	`let`/`const`
Reassigning a `const` by accident	runtime error	use `let` when you reassign

Mistake	Why it hurts	Best practice
`NaN === NaN` checks	always false	`Number.isNaN()`
Using ``		`for defaults with 0/""` wrong fallback `??`
Concatenating numbers as strings	`"1" + 2`	convert explicitly with `Number()`

- Prefer `Number.parseInt`/`Number.parseFloat` over `parseInt`.
- `let` by default, `const` whenever you can, `var` never.

2.5 Practice exercises

1. What do `"10" + 5` and `"10" - 5` produce?
2. Write a comparison that is false but true with `==`.
3. Give a default value for a possibly `0` setting.
4. Which values are falsy?
5. What does `typeof null` return, and why is it a trap?

Solutions

1. `"105"` and `5`.
2. `0 == false` is `true` while `0 === false` is `false`.
3. `settings ?? 10` (not `settings || 10`, which would also replace `0`).
4. `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`.
5. `"object"`, a historical bug kept for compatibility. It breaks `typeof x === "object"` checks for null, so test `x === null` explicitly.

2.6 Mini test

1. Multiple choice: how many primitive types does JavaScript have? - a) 5 - b) 6 - c) 7 - d) 8
2. True or false: `const` prevents changing the contents of an array.
3. Multiple choice: what is `"5" - 2`? - a) `"52"` - b) `3` - c) `NaN` - d) `"3"`
4. Short code question: build a greeting from two variables with a template literal.
5. Multiple choice: which operator only checks `null` and `undefined`? - a) `||` - b) `??` - c) `&&` - d) `===`
6. True or false: `NaN === NaN` is `true`.
7. Short answer: why prefer `===` over `==`?

Answer key and explanations

1. c - string, number, boolean, null, undefined, bigint, symbol.
2. False - `const` protects the binding; `push` still works.
3. b - `3`, because `-` converts both operands to numbers.

4. const hello = `Hello, ${name}`;
5. b - `??`.
6. False - `NaN` is not equal to anything, including itself; use `Number.isNaN()`.
7. Because `==` coerces operands with rules that surprise everyone (`0 == false`, `"" == 0`, `null == undefined`), while `===` compares type first and never converts.

Chapter 3 - Control flow and functions

3.1 Summary

- `if / else if / else` branches; `? :` is a compact conditional expression.
- Loops: `for`, `for...of`, `while`, `do...while`, and `for...in` for objects.
- `switch` compares with `===` and falls through without `break`.
- Functions are values: they can be stored, passed and returned.
- Arrow functions have no own `this` and no `arguments`.
- Default parameters, rest parameters and spread arguments cover most cases.
- Return early to keep functions shallow.
- A function expression is not hoisted like a declaration.
- `Array.prototype` methods often beat writing a loop.
- Recursion solves problems defined in terms of themselves.

3.2 Concepts

Branching

JavaScript

```javascript
const grade = 82;
if (grade >= 90) console.log("A");
else if (grade >= 80) console.log("B");
else console.log("C");

const label = grade >= 80 ? "pass" : "fail";
```

Loops

JSREF

```javascript
for (let i = 0; i < 3; i++) console.log(i);        // 0 1 2
for (const v of [10, 20]) console.log(v);           // 10 20
while (n > 0) n--;
for (const key in { a: 1, b: 2 }) console.log(key); // a b
```

`for...of` iterates values; `for...in` iterates keys of an object.

Function forms

JavaScript

```javascript
function declared() {}          // hoisted
const expression = function () {};  // not hoisted
const arrow = () => {};         // no own this, no arguments
const asyncFn = async () => {}; // always returns a promise
function* gen() { yield 1; }    // lazy sequence
```

Parameters

JavaScript

```javascript
function greet(name = "world", ...rest) {
  console.log(`Hello, ${name}`, rest);
}
greet("Ada", 1, 2);
```

output

```
Hello, Ada [ 1, 2 ]
```

3.3 Examples

Example 1: branching and the ternary.

JavaScript

```javascript
const price = 50;
const discount = price > 40 ? 10 : 0;
console.log(price - discount, discount ? "discounted" : "full price");
```

output

```
40 discounted
```

Example 2: the three loops you will actually use.

JavaScript

```javascript
const items = ["a", "b", "c"];

for (let i = 0; i < items.length; i++) console.log(i, items[i]);
for (const item of items) console.log(item);
items.forEach((item, i) => console.log(i, item));
```

output

```
0 a
1 b
2 c
a
b
c
0 a
1 b
2 c
```

Example 3: `while` and a counter.

JavaScript

```javascript
let n = 3;
let total = 0;
while (n > 0) {
  total += n;
  n--;
}
console.log(total, n);
```

output

```
6 0
```

Example 4: functions as values.

JavaScript

```javascript
const double = (n) => n * 2;
const apply = (fn, value) => fn(value);
console.log(apply(double, 21));
console.log([1, 2, 3].map(double).join(","));
```

output

```
42
2,4,6
```

Example 5: recursion.

JavaScript

```javascript
function factorial(n) {
  if (n <= 1) return 1;      // base case
  return n * factorial(n - 1);
}
console.log(factorial(5));
```

output

```
120
```

Example 6: default and rest parameters.

JavaScript

```javascript
function sum(a = 0, b = 0, ...rest) {
  return a + b + rest.reduce((x, y) => x + y, 0);
}
console.log(sum(), sum(1), sum(1, 2), sum(1, 2, 3, 4));
```

output

```
0 1 3 10
```

Example 7: `switch` and fall-through.

JavaScript

```
function dayType(day) {
  switch (day) {
    case "sat":
    case "sun":
      return "weekend";
    default:
      return "weekday";
  }
}
console.log(dayType("sat"), dayType("mon"));
```

output

```
weekend weekday
```

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`for...in` on an array	iterates keys, and inherited ones	`for...of` or index loop
`break` forgotten in `switch`	silent fall-through	every case ends with `return`/`break`
Off-by-one in `i <= length`	`undefined` access	`i < length`
Arrow functions everywhere	no `this` when you need it	regular methods for objects
Deep nesting	unreadable	early `return`
Recursion without a base case	stack overflow	always define the base case

- Use array methods (`map`, `filter`, `reduce`) instead of manual loops.
- Name functions for what they return, not what they do internally.

3.5 Practice exercises

1. Write a function that clamps a number between 0 and 100.
2. Print the keys of `{a: 1, b: 2}`.
3. Sum an array without a loop.
4. What is wrong with `for (const i = 0; ...)`?
5. Write an arrow function that returns the larger of two numbers.

Solutions

1. `js function clamp(n) { if (n < 0) return 0; if (n > 100) return 100; return n; }`
2. `for (const key in { a: 1, b: 2 }) console.log(key);`
3. `[1, 2, 3].reduce((a, b) => a + b, 0)`
4. `const i = 0` cannot be incremented: the binding is constant. Use `let`.

5. `const max = (a, b) => (a > b ? a : b);`

3.6 Mini test

1. Multiple choice: which loop iterates over values of an array? - a) `for...in` - b) `for...of` - c) `for` - d) `while`
2. True or false: arrow functions have their own `this`.
3. Multiple choice: what does `[1,2,3].reduce((a,b)=>a+b, 0)` return? - a) `[1,2,3]` - b) `6` - c) `0` - d) a function
4. Short code question: a function with a default `name` of `"world"`.
5. Multiple choice: which is hoisted? - a) function expressions - b) function declarations - c) `const` arrows - d) nothing
6. True or false: `switch` uses strict comparison.
7. Short answer: why is `i <= items.length` in a `for` loop a bug?

Answer key and explanations

1. b - `for...of`.
2. False - they take `this` from the enclosing scope, which is exactly why they are wrong as object methods.
3. b - `6`.
4. `function greet(name = "world") { return "Hello, " + name; }`
5. b - function declarations are hoisted with their body.
6. True - `case` comparisons use `===`.
7. On the last iteration `i` equals `items.length`, so `items[i]` is `undefined` and the loop does one iteration too many.

Chapter 4 - Objects, prototypes and classes

4.1 Summary

- Objects are collections of properties; values and getters/setters are possible.
- Property access: dot notation and bracket notation (for dynamic keys).
- Objects are copied by reference; spread or `structuredClone` for a copy.
- `this` is the object a method is called on, not where it was defined.
- Every object has a prototype, forming a chain that ends at `null`.
- Classes are syntactic sugar over prototypes with real inheritance.
- `Object.keys/values/entries/freeze/seal` manipulate objects as data.
- Private fields use `#name`; static members use `static`.

- `instanceof` checks the prototype chain; `in` checks for a property.
- Prototypes explain why arrays have `.map` without you adding it.

4.2 Concepts

Creating and reading

JavaScript

```javascript
const user = {};                      // empty object
user.name = "Ada";                    // add
user["full name"] = "Ada Lovelace";   // bracket for odd keys
console.log(user.name, user["full name"], user.missing);
```

output

```
Ada Ada Lovelace undefined
```

`this`

JavaScript

```javascript
const counter = {
  count: 0,
  increment() { this.count += 1; return this; },
};
counter.increment().increment();
console.log(counter.count);
```

output

```
2
```

Arrow functions capture `this` from the enclosing scope, so they are wrong for methods but right for callbacks.

Prototype chain

JavaScript

```javascript
const animal = { eats: true };
const rabbit = Object.create(animal);
rabbit.jumps = true;
console.log(rabbit.eats, rabbit.jumps);
console.log(Object.getPrototypeOf(rabbit) === animal);
```

output

```
true true
true
```

Classes

JavaScript

```javascript
class Temperature {
  #celsius = 0;
  get celsius() { return this.#celsius; }
  get fahrenheit() { return this.#celsius * 1.8 + 32; }
  set fahrenheit(value) { this.#celsius = (value - 32) / 1.8; }
}

const t = new Temperature();
t.fahrenheit = 212;
console.log(t.fahrenheit, t.celsius);
```

output

```
212 100
```

A getter is read like a property, a setter is assigned like a property, and both compute from a private field that nothing outside the class can touch.

Object utilities

JSREF

```javascript
const frozen = Object.freeze({ a: 1 });
frozen.a = 2;                    // silently ignored (throws in strict mode)
console.log(frozen.a);
console.log("a" in frozen, Object.keys(frozen).join(","));
```

output

```
1
a
```

4.3 Examples

Example 1: references versus copies.

JavaScript

```javascript
const original = { a: 1, nested: { b: 2 } };
const alias = original;                 // same object
const shallow = { ...original };        // one level
const deep = structuredClone(original); // recursive

alias.a = 10;
shallow.nested.b = 20;
deep.nested.b = 30;

console.log(original.a, original.nested.b, deep.nested.b);
```

output

```
10 20 30
```

Example 2: destructuring with defaults and renaming.

JavaScript

```javascript
const user = { name: "Ada", role: "admin", meta: { id: 7 } };
const { name, role = "user", missing = "default", meta: { id } } = user;
console.log(name, role, missing, id);
```

output

```
Ada admin default 7
```

Example 3: getters and setters.

JavaScript

```javascript
class Temperature {
  #celsius = 0;
  get fahrenheit() { return this.#celsius * 1.8 + 32; }
  set fahrenheit(value) { this.#celsius = (value - 32) / 1.8; }
}

const t = new Temperature();
t.fahrenheit = 212;
console.log(t.fahrenheit);
```

output

```
212
```

Example 4: `Object.entries` round trip.

JavaScript

```javascript
const scores = { html: 90, css: 85 };
const doubled = Object.fromEntries(
  Object.entries(scores).map(([key, value]) => [key, value * 2])
);
console.log(JSON.stringify(doubled));
```

output

```
{"html":180,"css":170}
```

Example 5: instanceof and `in`.

JavaScript

```javascript
class A {}
class B extends A {}
const b = new B();
console.log(b instanceof A, b instanceof B, b instanceof Object);
console.log("constructor" in b);
```

output

```
true true true
true
```

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Assigning an object to "copy" it	mutations leak	spread or `structuredClone`
Arrow function as a method	`this` is wrong	regular function syntax
Deep `#private` everywhere	hard to test	expose a small public API
Modifying built-in prototypes	breaks everything	extend or use helpers
`Object.freeze` in the constructor	hard to evolve	freeze only constants
Using classes for stateless helpers	overkill	plain functions and objects

- `Object.freeze` is a promise, not a guarantee; it only affects own properties.
- Prefer composition (objects holding objects) to deep inheritance chains.

4.5 Practice exercises

1. Copy an object without sharing its nested property.
2. Write a class `Point` with a private `x`, a getter `x` and a `move(dx)` method.
3. Why is `const copy = original` not a copy?
4. Convert `{a:1,b:2}` into `[["a",1],["b",2]]`.
5. What does `this` refer to inside `setTimeout(() => this.x, 0)` on an object?

Solutions

1. `const copy = structuredClone(original);` (or `{...original}` if it is flat).
2. `js class Point { #x = 0; get x() { return this.#x; } move(dx) { this.#x += dx; return this; } }`
3. It copies the reference: both names point to the same object, so changing one changes the other.
4. `Object.entries({ a: 1, b: 2 })`
5. The arrow function takes `this` from the enclosing scope, so it is the object that defined the method (unlike a plain function, which would lose `this`).

4.6 Mini test

1. Multiple choice: which creates a real copy of a nested object? - a) `Object.assign({}, o)` - b) `{ ...o }` - c) `structuredClone(o)` - d) `o.slice()`
2. True or false: arrays inherit their methods from `Array.prototype`.
3. Multiple choice: which syntax declares a private field? - a) `_x` - b) `#x` - c) `private x` - d) `x!`
4. Short code question: read `user.name` destructured with a fallback.
5. Multiple choice: what does `instanceof` test? - a) the constructor by name - b) the prototype chain - c) the type of the value - d) nothing
6. True or false: `Object.freeze` throws in strict mode when you write to a frozen property.

7. Short answer: why avoid arrow functions for object methods?

Answer key and explanations

1. c - `structuredClone` is recursive; the others copy one level only.
2. True - that is why `[].map` works on any array without adding it.
3. b - `#x`.
4. `const { name = "anonymous" } = user;`
5. b - it walks the prototype chain looking for the prototype.
6. True - in strict mode (modules and classes) the assignment throws.
7. Because an arrow function has no own `this`, so a method written as an arrow will read `this` from the surrounding scope instead of the instance.

Chapter 5 - Strings, numbers and arrays

5.1 Summary

- Strings are immutable; every method returns a new string.
- Template literals, `slice`, `split`, `replace`, `includes` cover most needs.
- Numbers are doubles; use `toFixed`, `Number.parseInt`, `Number.parseFloat`.
- `NaN` appears when a number conversion fails; check with `Number.isNaN`.
- Arrays are objects with numeric keys and a `length` that grows automatically.
- `map`, `filter`, `reduce` replace most loops.
- `find`/`findIndex` stop early; `some`/`every` return booleans.
- `sort` sorts lexicographically by default: pass a comparator for numbers.
- `flat`, `at` and spread make modern array code shorter.
- Strings and arrays are copied by value and reference respectively.

5.2 Concepts

String methods

Method	Result
`slice(start, end)`	the substring between the indices
`substring` / `substr`	legacy alternatives
`indexOf` / `lastIndexOf`	position or `-1`
`includes` / `startsWith` / `endsWith`	booleans
`toUpperCase` / `toLowerCase` / `trim` / `padStart`	transformed text
`split(sep)`	an array

Method	Result
`replace` / `replaceAll`	a new string
`repeat` / `concat`	duplication or joining

Number methods

JSREF

```
Number.parseInt("12px", 10)   // 12
Number.parseFloat("1.5kg")   // 1.5
(1.005).toFixed(2)           // "1.00" (floating point!)
Number.isInteger(5)          // true
Number.isFinite(1 / 0)        // false
(255).toString(16)           // "ff"
```

Array methods that matter

Method	Returns
`push` / `pop`	the new length / the removed element
`shift` / `unshift`	the removed element / the new length
`slice`	a shallow copy of a range
`splice`	the removed elements (mutates)
`concat` / `flat`	a new array
`indexOf` / `includes`	a position or a boolean
`map` / `filter` / `reduce`	a new array or a value
`find` / `findIndex`	an element or its index
`some` / `every`	a boolean
`sort` / `reverse`	the same array, sorted / reversed (mutate)

5.3 Examples

Example 1: array transformation.

JavaScript

```
const nums = [1, 2, 3, 4, 5];
console.log(nums.map((n) => n * 2).join(","));
console.log(nums.filter((n) => n % 2 === 1).join(","));
console.log(nums.reduce((sum, n) => sum + n, 0));
console.log(nums.find((n) => n > 3), nums.findIndex((n) => n > 3));
console.log(nums.some((n) => n > 4), nums.every((n) => n > 0));
```

output

```
2,4,6,8,10
1,3,5
15
4 3
true true
```

Example 2: sorting with a comparator.

JavaScript

```javascript
console.log([10, 9, 80, 100].sort().join(","));
console.log([10, 9, 80, 100].sort((a, b) => a - b).join(","));
console.log(["b", "a", "c"].sort().join(","));
const byAge = (a, b) => a.age - b.age;
console.log([{ age: 30 }, { age: 10 }].sort(byAge).map((o) => o.age).join(","));
```

output

```
10,100,80,9
9,10,80,100
a,b,c
10,30
```

Example 3: string toolbox.

JavaScript

```javascript
const text = "  Hello, JavaScript world  ";
console.log(text.trim());
console.log(text.trim().slice(0, 5));
console.log(text.trim().replaceAll("o", "0"));
console.log(text.includes("Java"), text.trim().startsWith("Hello"));
console.log(text.trim().split(", ").length);
console.log("ab".repeat(3), "7".padStart(3, "0"));
```

output

```
Hello, JavaScript world
Hello
Hell0, JavaScript w0rld
true true
2
ababab 007
```

Example 4: number toolbox.

JavaScript

```javascript
console.log((1234.5678).toFixed(2));
console.log(Number.parseInt("42px", 10), Number.parseFloat("3.5kg"));
console.log(Number.isNaN(NaN), Number.isNaN("x"), Number.isInteger(5));
console.log((255).toString(16), (0.1 + 0.2));
console.log(Math.max(1, 2, 3), Math.round(2.5), Math.floor(-2.5));
console.log(Math.abs(-7), Math.sqrt(16), 2 ** 10);
```

output

```
1234.57
42 3.5
true false true
ff 0.30000000000000004
3 3 -3
7 4 1024
```

Example 5: copy, splice, flat and at.

JavaScript

```javascript
const base = [1, 2, 3, 4];
console.log(base.slice(1, 3).join(","));
console.log(base.at(-1), base.at(0));
console.log([1, [2, [3]]].flat(Infinity).join(","));
const removed = base.splice(1, 2);
console.log(removed.join(","), base.join(","));
```

output

```
2,3
4 1
1,2,3
2,3 1,3,4
```

Example 6: grouping with reduce.

JavaScript

```javascript
const words = ["ant", "bee", "ape", "cat"];
const byLetter = words.reduce((acc, w) => {
  const k = w[0];
  acc[k] = (acc[k] || []).concat(w);
  return acc;
}, {});
console.log(JSON.stringify(byLetter));
```

output

```
{"a":["ant","ape"],"b":["bee"],"c":["cat"]}
```

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`sort()` on numbers	lexicographic order	pass a comparator
Mutating with `sort`/`reverse`	surprises the caller	copy first, then sort
`for (let i...)` to transform	verbose	`map`/`filter`/`reduce`
`+` with a number and a string	string concatenation	`Number()` or template literals
`toFixed` for money	rounding surprises	integer cents or a decimal library
Comparing floats with `===`	precision	compare with an epsilon
`parseInt` without a radix	legacy surprises	`Number.parseInt(s, 10)`

- Treat arrays as immutable in shared code: `const next = [...base, item]`.
- Chain array methods; one transformation per line reads better.

5.5 Practice exercises

1. Sort `[5, 20, 100]` numerically.
2. Count how many words in an array are longer than three letters.
3. Return the unique values of `[1, 2, 2, 3]`.
4. Extract the numbers from `"a1b22c333"`.
5. Why does `[10, 9].sort()` give `[10, 9]`?

Solutions

1. `[5, 20, 100].sort((a, b) => a - b)`
2. `js ["one", "four", "six", "a"].filter((w) => w.length > 3).length`
3. `[...new Set([1, 2, 2, 3])]`
4. `"a1b22c333".match(/\d+/g)`
5. Because `sort` converts to strings and compares them lexicographically, and `"10" < "9"` as text. A numeric comparator fixes it.

5.6 Mini test

1. Multiple choice: which method returns a new array? - a) `push` - b) `sort` - c) `map` - d) `splice`
2. True or false: strings are mutable in JavaScript.
3. Multiple choice: what does `"abc".slice(1)` return? - a) `"a"` - b) `"bc"` - c) `"abc"` - d) `"b"`
4. Short code question: sum `[1,2,3]` with `reduce`.
5. Multiple choice: what does `0.1 + 0.2` print? - a) `0.3` - b) `0.30000000000000004` - c) `0.3000` - d) `NaN`
6. True or false: `Number.isNaN("x")` is `true`.
7. Short answer: which two methods return booleans for arrays?

Answer key and explanations

1. c - `map` returns a new array; the others mutate or return a length.
2. False - strings are immutable, so every method builds a new string.
3. b - `"bc"`.
4. `[1, 2, 3].reduce((a, b) => a + b, 0)` returns `6`.
5. b - binary floating point cannot represent `0.3` exactly.
6. False - `"x"` is not `NaN`; `Number("x")` is.
7. `some` and `every`.

Chapter 6 - Iterables, Map, Set, destructuring, JSON

6.1 Summary

- `Map` keeps keys of any type and remembers insertion order.
- `Set` stores unique values and removes duplicates.
- `for...of` works on anything iterable: arrays, strings, Maps, Sets.
- Spread `...` turns any iterable into an array.
- Destructuring binds array positions and object properties to variables.
- `Object.keys/values/entries` turn objects into arrays of data.
- `JSON.stringify` and `JSON.parse` move data across boundaries.
- Generators produce values lazily with `yield`.
- `structuredClone` deep-copies data without losing types.
- `Map` beats plain objects when keys are unknown or dynamic.

6.2 Concepts

Map

JavaScript

```javascript
const m = new Map([["a", 1]]);
m.set("b", 2);
m.get("a"); m.has("c"); m.delete("b"); m.size; m.clear();
for (const [key, value] of m) console.log(key, value);
```

Set

JavaScript

```javascript
const ids = new Set([1, 1, 2]);
ids.add(3); ids.delete(1); ids.has(2); [...ids];
```

Iterables

An object is iterable when it has a `[Symbol.iterator]` method that returns an iterator with a `next()` method. Arrays, strings, Maps, Sets, `arguments`, `NodeList` and generators are all iterable.

Destructuring

JSREF

```
const [a = 0, ...rest] = [1, 2, 3];        // arrays
const { x, y: renamed, z = 9 } = obj;      // objects, rename, default
const { a: { b } } = { a: { b: 1 } };      // nested
let p = 1, q = 2;
[p, q] = [q, p];                            // swap
```

JSON

Direction	Function	Notes
value to text	`JSON.stringify(v)`	drops `undefined`, functions, symbols
text to value	`JSON.parse(s)`	throws on invalid input

`JSON.parse` never runs code, unlike `eval`, which is why it is safe for data.

6.3 Examples

Example 1: Map and Set.

JavaScript

```
const map = new Map();
map.set("html", 90).set("css", 85);
console.log(map.size, map.get("css"), map.has("js"));
console.log([...map.keys()].join(","), [...map.values()].join(","));

const set = new Set([1, 1, 2, 3, 3]);
console.log(set.size, [...set].join(","));
```

output

```
2 85 false
html,css 90,85
3 1,2,3
```

Example 2: unique values and iteration.

JavaScript

```
const words = ["ada", "alan", "ada", "grace"];
const unique = [...new Set(words)];
console.log(unique.join(","));
for (const [i, w] of unique.entries()) console.log(i, w);
```

output

```
ada,alan,grace
0 ada
1 alan
2 grace
```

Example 3: destructuring.

JavaScript

```javascript
const [first, second, ...others] = [1, 2, 3, 4];
console.log(first, second, others.join(","));

const user = { name: "Ada", address: { city: "London" } };
const { name, address: { city }, zip = "00000" } = user;
console.log(name, city, zip);

let p = 1, q = 2;
[p, q] = [q, p];
console.log(p, q);
```

output

```
1 2 3,4
Ada London 00000
2 1
```

Example 4: objects as data.

JavaScript

```javascript
const scores = { html: 90, css: 85, js: 88 };
console.log(Object.keys(scores).join(","));
console.log(Object.values(scores).join(","));
console.log(Object.entries(scores).map(([k, v]) => k + ":" + v).join(" "));
console.log(JSON.stringify(scores));
```

output

```
html,css,js
90,85,88
html:90 css:85 js:88
{"html":90,"css":85,"js":88}
```

Example 5: JSON round trip.

JavaScript

```javascript
const text = '{"name":"Ada","tags":["math","code"],"active":true}';
const user = JSON.parse(text);
console.log(user.name, user.tags.length, user.active);
const out = JSON.stringify(user, ["name", "active"]);
console.log(out);
console.log(JSON.stringify({ a: undefined, b: 1 }));
```

output

```
Ada 2 true
{"name":"Ada","active":true}
{"b":1}
```

Example 6: generators.

JavaScript

```
function* range(n) {
  for (let i = 0; i < n; i++) yield i;
}
console.log([...range(4)].join(","));

function* counter() {
  let i = 0;
  while (true) yield i++;
}
const c = counter();
console.log(c.next().value, c.next().value, c.next().value);
```

output

```
0,1,2,3
0 1 2
```

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`obj[key]` counting on a Map	prototype keys collide	use `Map`
Iterating a Map with `for...in`	nothing, Maps are not enumerable that way	`for...of`
`JSON.stringify` on a Map	empty object	convert with `Object.fromEntries`
`eval` for JSON	code execution	`JSON.parse`
Deep clone with spread on nested data	shared references	`structuredClone`
Ignoring iterator protocol	reinventing loops	implement `[Symbol.iterator]`

- Use `[...set]` and `[...map.keys()]` to bridge to arrays.
- Generators make lazy pipelines cheap: nothing runs until you ask.

6.5 Practice exercises

1. Get the unique characters of `"mississippi"`.
2. Count how many times each word appears in an array, using a `Map`.
3. Destructure `{ id: 5 }` with a default label.
4. Convert a `Map` to an object.
5. What does `JSON.stringify([])` and `JSON.stringify([undefined])` return?

Solutions

1. `[...new Set("mississippi")].join("")` gives `"misp"`.

2. `js const counts = new Map(); for (const w of words) counts.set(w, (counts.get(w) || 0) + 1);`
3. `const { id, label = "none" } = { id: 5 };`
4. `Object.fromEntries(map)`
5. `"[]"` and `"[null]"`: `undefined` becomes `null` inside arrays and is dropped in objects.

6.6 Mini test

1. Multiple choice: which structure guarantees unique values? - a) `Map` - b) `Set` - c) `Object` - d) `Array`
2. True or false: `Map` accepts objects as keys.
3. Multiple choice: which syntax turns any iterable into an array? - a) `Array.from(...)` - b) `[...iterable]` - c) both - d) neither
4. Short code question: read `obj.city` from a nested `address` with destructuring.
5. Multiple choice: what does `JSON.parse('{"a":1}')` return? - a) a string - b) an object - c) an array - d) it throws
6. True or false: generators compute their values lazily.
7. Short answer: why is `JSON.parse` safer than `eval`?

Answer key and explanations

1. b - `Set`.
2. True - keys may be any type, including objects and functions.
3. c - both work; spread is usually shorter.
4. `const { address: { city } } = obj;`
5. b - an object with `a === 1`.
6. True - the body runs only when `next()` is called.
7. `JSON.parse` only understands JSON syntax, so it can never execute code even if the input is malicious, while `eval` runs any JavaScript it is given.

Chapter 7 - Scope, closures and advanced functions

7.1 Summary

- Scope is lexical: where you write a function decides what it can see.
- `var` is function-scoped and hoisted; `let`/`const` are block-scoped.
- A closure keeps its outer variables alive after the outer function returns.
- Closures power counters, memoisation, modules and event handlers.
- Every function call creates a new scope; `this` depends on the call.

- `call`, `apply` and `bind` set `this` explicitly.
- Rest parameters collect arguments; spread expands them.
- Currying turns a multi-argument function into single-argument steps.
- `arguments` is legacy; use rest parameters instead.
- Closures hold memory, so avoid keeping huge objects alive by accident.

7.2 Concepts

Scope in one example

JavaScript

```javascript
let x = "outer";
function outer() {
  let x = "inner";
  function inner() { return x; }
  return inner();
}
console.log(outer(), x);
```

output

```
inner outer
```

The inner `x` shadows the outer one, and the inner function resolves to the nearest declaration when it runs.

Closure

JavaScript

```javascript
function makeAdder(x) {
  return (y) => x + y;      // keeps x alive
}
const addFive = makeAdder(5);
console.log(addFive(2), addFive(10));
```

output

```
7 15
```

`this`

Call style	this
`obj.method()`	`obj`
`fn()`	`undefined` (strict) or the global object
arrow function	the enclosing scope's `this`
`fn.call(x)` / `fn.apply(x)`	`x`
`fn.bind(x)()`	`x`

Call style	this
`new Fn()`	the new object

Currying and composition

JavaScript

```javascript
const add = (a) => (b) => (c) => a + b + c;
const compose = (f, g) => (x) => f(g(x));
const inc = (n) => n + 1;
const double = (n) => n * 2;
console.log(compose(inc, double)(5));
```

output

```
11
```

`double(5)` is `10`, then `inc(10)` is `11`.

7.3 Examples

Example 1: a private counter.

JavaScript

```javascript
function createCounter() {
  let count = 0;
  return {
    increment: () => ++count,
    get value() { return count; },
  };
}
const c = createCounter();
c.increment();
c.increment();
console.log(c.value, createCounter().value);
```

output

```
2 0
```

Example 2: memoisation with a closure.

JavaScript

```javascript
function memoSquare() {
  const cache = new Map();
  return (n) => {
    if (cache.has(n)) return cache.get(n) + " (cached)";
    const result = n * n;
    cache.set(n, result);
    return String(result);
  };
}
const square = memoSquare();
console.log(square(4), square(4), square(5));
```

output

```
16 (cached) 25
```

Example 3: closures in loops.

JavaScript

```js
const fns = [];
for (let i = 0; i < 3; i++) fns.push(() => i);
console.log(fns.map((f) => f()).join(","));

const varFns = [];
for (var j = 0; j < 3; j++) varFns.push(() => j);
console.log(varFns.map((f) => f()).join(","));
```

output

```
0,1,2
3,3,3
```

`let` creates a binding per iteration; `var` shares one, so every closure sees the final value.

Example 4: `call`, `apply`, `bind`.

JavaScript

```js
const obj = { name: "Ada" };
function greet(tag) { return tag + " " + this.name; }
console.log(greet.call(obj, "call:"));
console.log(greet.apply(obj, ["apply:"]));
const bound = greet.bind(obj, "bind:");
console.log(bound());
```

output

```
call: Ada
apply: Ada
bind: Ada
```

Example 5: rest and spread.

JavaScript

```js
function collect(...items) { return items.length + ":" + items.join("-"); }
const parts = ["a", "b", "c"];
console.log(collect(...parts));
const merged = [...[1, 2], ...[3, 4]];
console.log(JSON.stringify(merged));
```

output

```
3:a-b-c
[1,2,3,4]
```

Example 6: currying.

JavaScript

```
const curry = (a) => (b) => (c) => a + b + c;
console.log(curry(1)(2)(3));
const withTwo = curry(2);
console.log(withTwo(3)(4), withTwo(5)(6));
```

output

```
6
9 13
```

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`var` in loops	shared binding, stale closures	`let`
Detaching a method that uses `this`	`this` becomes `undefined`	bind it, or use an arrow wrapper
Keeping big objects in closures	memory leaks	release references when done
`arguments` in new code	not an array, legacy	rest parameters
Arrow methods in classes	`this` errors	regular methods
Deep currying for its own sake	unreadable	only where it helps reuse

- A closure is a function plus the scope where it was created: that is the whole idea.
- Use closures to make state private instead of prefixing with _.

7.5 Practice exercises

1. Write a function that returns a function multiplying by a fixed factor.
2. Why do all three functions in a `var` loop log the same number?
3. Fix a detached method that loses `this`.
4. Write a `sum` that accepts any number of arguments.
5. What does the closure keep alive after the outer function returns?

Solutions

1. `js const multiplyBy = (factor) => (n) => n * factor; const triple = multiplyBy(3);`
2. `var` has one binding for the whole function, so every closure reads the same variable, whose final value is the loop's exit value. `let` gives each iteration its own binding.
3. Bind it: `const bound = obj.method.bind(obj);` or define the callback as an arrow function inside the method.
4. `js const sum = (...nums) => nums.reduce((a, b) => a + b, 0);`
5. The variables of the outer scope it references: here the private `count`, so the state survives as long as some closure can still reach it.

7.6 Mini test

1. Multiple choice: what is a closure? - a) a function inside a function - b) a function plus its lexical scope - c) a private class field - d) a callback
2. True or false: `let` in a loop gives each iteration its own binding.
3. Multiple choice: which method sets `this` permanently? - a) `call` - b) `apply` - c) `bind` - d) `of`
4. Short code question: a function that returns the sum of any arguments.
5. Multiple choice: what is `this` inside a plain function called with no receiver in strict mode? - a) the global object - b) `undefined` - c) `window` - d) `null`
6. True or false: `arguments` is a real array.
7. Short answer: name one risk of closures.

Answer key and explanations

1. b - a function together with the environment it was created in.
2. True - that is why per-iteration closures work with `let`.
3. c - `bind` returns a new function with a fixed `this`.
4. `const sum = (...nums) => nums.reduce((a, b) => a + b, 0);`
5. b - `undefined` in strict mode.
6. False - it is array-like; use rest parameters or `[...arguments]`.
7. Memory: the closure keeps its captured variables alive, so forgetting to release a large object can leak.

Chapter 8 - Error handling

8.1 Summary

- `try / catch / finally` handles exceptions without crashing the program.
- `throw` raises any value; `throw new Error(...)` is the convention.
- `Error` has `name`, `message` and (in engines) `stack`.
- `JSON.parse` and `JSON.stringify` throw on invalid input or cycles.
- Custom errors extend `Error` and set their own `name`.
- `catch` without a binding catches everything, including programmer errors.
- `finally` always runs: cleanup code belongs there.
- Errors propagate up the call stack until a `try` catches them.
- Unhandled promise rejections warn (or crash) instead of throwing.
- Fail fast: validate input and throw early, close to the cause.

8.2 Concepts

The syntax

JSREF

```
try {
  risky();
} catch (error) {
  console.error(error.name, error.message);
} finally {
  cleanup();
}
```

Throwing

JavaScript

```
function requireAdult(age) {
  if (typeof age !== "number") throw new TypeError("age must be a number");
  if (age < 18) throw new RangeError("too young");
  return true;
}
```

Built-in error types

Type	Cause
`Error`	the base
`TypeError`	wrong type: `null.x`, `undefined()`
`RangeError`	a value outside the allowed range
`SyntaxError`	invalid syntax at parse time
`ReferenceError`	a variable that is not defined
`EvalError`, `URIError`	legacy
`AggregateError`	several errors at once (`Promise.any`)

Strict mode turns mistakes into errors

JSREF

```
"use strict";
undeclared = 1;   // ReferenceError instead of a silent global
```

8.3 Examples

Example 1: catching a `TypeError`.

JavaScript

```javascript
try {
  null.x;
} catch (error) {
  console.log(error instanceof TypeError, error.name);
  console.log(error.message);
} finally {
  console.log("cleanup");
}
```

output

```
true TypeError
Cannot read properties of null (reading 'x')
cleanup
```

Example 2: throwing with a guard.

JavaScript

```javascript
function requireAdult(age) {
  if (typeof age !== "number") throw new TypeError("age must be a number");
  if (age < 18) throw new RangeError("too young");
  return true;
}

for (const value of [20, 15, "x"]) {
  try {
    requireAdult(value);
    console.log(value, "ok");
  } catch (error) {
    console.log(value, error.name, error.message);
  }
}
```

output

```
20 ok
15 RangeError too young
x TypeError age must be a number
```

Example 3: a custom error.

JavaScript

```javascript
class ValidationError extends Error {
  constructor(message, field) {
    super(message);
    this.name = "ValidationError";
    this.field = field;
  }
}

try {
  throw new ValidationError("is required", "email");
} catch (error) {
  console.log(error.name, error.message, error.field, error instanceof Error);
}
```

output

```
ValidationError is required email true
```

Example 4: `JSON.parse` failure handling.

JavaScript

```
function readJson(text) {
  try {
    return JSON.parse(text);
  } catch {
    return null;
  }
}
console.log(readJson('{"a":1}'), readJson("{oops}"));
```

output

```
[object Object] null
```

Example 5: an error inside a function propagates.

JavaScript

```
function a() { return b(); }
function b() { throw new Error("from b"); }

try {
  a();
} catch (error) {
  console.log("caught:", error.message);
}
```

output

```
caught: from b
```

Example 6: rethrowing unknown errors.

JavaScript

```
function handle(fn) {
  try {
    return fn();
  } catch (error) {
    if (error instanceof RangeError) return "range problem";
    throw error;              // let real bugs surface
  }
}
console.log(handle(() => { throw new RangeError("x"); }));
try {
  handle(() => { throw new TypeError("bug"); });
} catch (error) {
  console.log("propagated:", error.name);
}
```

output

```
range problem
propagated: TypeError
```

8.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Catching everything silently	bugs vanish	rethrow what you do not handle
`catch (e) {}` empty block	errors disappear	log or rethrow
Throwing strings	no stack, no type	`throw new Error(...)`
Validating everywhere at the end	hard to trace	validate at the boundary
Using exceptions for normal flow	slow, unclear	return values or result objects
Swallowing async errors	silent failures	`await` inside `try`
`finally` with a `return`	overrides thrown errors	avoid returns in `finally`

- Log the error once, with context, and rethrow or convert it.
- Attach context when rethrowing: `error.cause = original`.

8.5 Practice exercises

1. Catch a `JSON.parse` failure and return a default value.
2. Write a custom error `NotFoundError` with a `resource` field.
3. What does `finally` guarantee?
4. Why is `throw "oops"` worse than `throw new Error("oops")`?
5. Use `error.cause` when wrapping an error.

Solutions

1. `js function read(text) { try { return JSON.parse(text); } catch { return {}; } }`
2. `js class NotFoundError extends Error { constructor(resource) { super(`${resource} not found`); this.name = "NotFoundError"; this.resource = resource; } }`
3. That the block runs whether or not the `try` block threw or returned, which makes it the right place for cleanup (`close()`, releasing locks).
4. A string has no `name`, no `message` and no `stack`, so logging and error handling lose information. `Error` also makes `instanceof` work.
5. `js throw new Error("loading failed", { cause: original });`

8.6 Mini test

1. Multiple choice: which block always runs? - a) `try` - b) `catch` - c) `finally` - d) `do`
2. True or false: `catch` can omit its parameter in modern JavaScript.
3. Multiple choice: what does reading `null.x` throw? - a) `RangeError` - b) `TypeError` - c) `SyntaxError` - d) nothing
4. Short code question: throw an error when a list is empty.
5. Multiple choice: which is the base class of all errors? - a) `Throwable` - b) `Error` - c) `Exception` - d) `BaseError`

6. True or false: an uncaught exception in a function stops the whole script.
7. Short answer: why rethrow errors you do not recognise?

Answer key and explanations

1. c - `finally`.
2. True - `catch { }` without a binding is supported since ES2019.
3. b - `TypeError`.
4. `js if (list.length === 0) throw new RangeError("list must not be empty");`
5. b - `Error`.
6. True - it propagates up the stack until a `try` catches it, and if none does, execution stops with the error in the console.
7. Because catching everything hides real bugs. Convert only the errors you understand and rethrow the rest so they stay visible.

Chapter 9 - Asynchronous JavaScript

9.1 Summary

- JavaScript runs one call stack; blocking it stops everything.
- Callbacks were the first async pattern; promises replaced them.
- A promise is pending, fulfilled or rejected: `then`, `catch`, `finally`.
- `Promise.all` waits for all, `any` for the first success, `race` for the first settlement.
- `async / await` is syntax over promises, and makes async code read as sync.
- `for await...of` consumes async iterables in order.
- The event loop drains microtasks (promises) before macrotasks (timers).
- `setTimeout` and `Promise` output order follows the loop, not the source order.
- Always handle rejections: an unhandled one warns or crashes the process.
- `AbortController` cancels `fetch` and removes listeners.

9.2 Concepts

The event loop

output

```
call stack -> microtasks (promise reactions, queueMicrotask)
           -> macrotasks (setTimeout, setInterval, I/O, events)
```

Every callback finishes before the next task starts, which is why "concurrency" in JavaScript is cooperative.

Promises

JavaScript

```javascript
const p = new Promise((resolve, reject) => {
  setTimeout(() => resolve("done"), 100);
});
p.then((v) => console.log(v)).catch((e) => console.error(e));
```

Static method	Behaviour
`Promise.resolve(v)`	an already fulfilled promise
`Promise.reject(e)`	an already rejected promise
`Promise.all(ps)`	all fulfilled, or the first rejection
`Promise.allSettled(ps)`	waits for every outcome
`Promise.any(ps)`	the first fulfilment, else `AggregateError`
`Promise.race(ps)`	the first settlement

`async` / `await`

JavaScript

```javascript
async function loadUser(id) {
  try {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) throw new Error("HTTP " + res.status);
    return await res.json();
  } catch (error) {
    console.error("load failed:", error.message);
    return null;
  }
}
```

An `async` function always returns a promise; `await` unwraps one.

9.3 Examples

Example 1: order of execution.

JavaScript

```javascript
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
```

output

```
1
4
3
2
```

The synchronous code runs first, then the microtask (`3`), then the timer (`2`).

Example 2: `then` chaining.

JavaScript

```javascript
Promise.resolve(1)
  .then((v) => v + 1)
  .then((v) => v * 10)
  .then((v) => console.log(v));
```

output

```
20
```

Example 3: `async`/`await` with `try/catch`.

JavaScript

```javascript
async function main() {
  const value = await Promise.resolve("ok");
  try {
    await Promise.reject(new Error("boom"));
  } catch (error) {
    console.log(value, error.message);
  }
}
main();
```

output

```
ok boom
```

Example 4: `Promise.all` and `allSettled`.

JavaScript

```javascript
Promise.all([Promise.resolve(1), Promise.resolve(2), Promise.resolve(3)])
  .then((values) => console.log(values.join(",")));

Promise.allSettled([Promise.resolve(1), Promise.reject(new Error("x"))])
  .then((results) => console.log(results.map((r) => r.status).join(",")));
```

output

```
1,2,3
fulfilled,rejected
```

Example 5: `async`/`await` with `all`.

JavaScript

```javascript
async function fetchAll(ids) {
  const users = await Promise.all(ids.map((id) => Promise.resolve({ id })));
  return users.map((u) => u.id).join(",");
}
fetchAll([3, 1, 2]).then(console.log);
```

output

```
3,1,2
```

Example 6: retries and timeouts.

JavaScript

```javascript
async function attempt(fn, retries = 2) {
  for (let i = 0; i <= retries; i++) {
    try {
      return await fn();
    } catch (error) {
      if (i === retries) throw error;
    }
  }
}

let calls = 0;
attempt(() => {
  calls += 1;
  return calls < 3 ? Promise.reject(new Error("nope")) : Promise.resolve("yes");
}).then((v) => console.log(v, calls)).catch((e) => console.log("failed", e.message));
```

output

```
yes 3
```

Example 7: async iterables.

JavaScript

```javascript
async function* ticker(n) {
  for (let i = 0; i < n; i++) yield i;
}
(async () => {
  const seen = [];
  for await (const value of ticker(3)) seen.push(value);
  console.log(seen.join(","));
})();
```

output

```
0,1,2
```

9.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Nested `then` callbacks	a pyramid of doom	`async / await`
Forgetting `await`	you handle a promise, not the value	always `await`
No rejection handler	silent failure / crash	`try/catch` or `.catch`
`forEach` with an async callback	does not wait	`for...of` or `Promise.all`
Sequential awaits for independent work	unnecessarily slow	`Promise.all`
Unbounded `Promise.all` on huge arrays	memory and rate limits	batches
Ignoring `res.ok`	errors look like data	check the status

- Prefer `async / await`, but know the promise API: `all`, `race`, `allSettled`.
- Wrap `await` in `try/catch` at the boundary where you can actually recover.

9.5 Practice exercises

1. Predict the output order of `log(1); setTimeout(log(2)); Promise.resolve().then(log(3)); log(4)`.
2. Await two independent requests at once.
3. Retry a promise-returning function up to three times.
4. Why does `[1,2,3].forEach(async ...)` not wait?
5. What happens to a rejected promise that nobody handles?

Solutions

1. `1 4 3 2`: synchronous first, then the microtask, then the timer.
2. `await Promise.all([fetch("/a"), fetch("/b")])`
3. `js async function retry(fn, n = 3) { for (let i = 0; i < n; i++) { try { return await fn(); } catch (e) { if (i === n - 1) throw e; } } }`
4. `forEach` ignores the returned promise, so the loop finishes immediately and the async callbacks run later. Use `for...of` to await each, or `Promise.all(items.map(...))`.
5. It becomes an unhandled rejection: Node logs a warning (and terminates in newer versions), while browsers log an error. Always attach a handler.

9.6 Mini test

1. Multiple choice: which runs before `setTimeout(..., 0)`? - a) I/O callbacks - b) promise reactions - c) `setInterval` - d) nothing
2. True or false: an `async` function returns a promise.
3. Multiple choice: which static method waits for every outcome? - a) `Promise.all` - b) `Promise.race` - c) `Promise.allSettled` - d) `Promise.any`
4. Short code question: await a promise and log its value in a function.
5. Multiple choice: what does `for await...of` need? - a) a generator - b) an async iterable - c) an array - d) a callback
6. True or false: `fetch` rejects on HTTP error status like 404.
7. Short answer: how do you run two independent requests at once?

Answer key and explanations

1. b - microtasks (promises) drain before timers.
2. True - even when it returns a plain value, it is wrapped in a promise.
3. c - `Promise.allSettled` never rejects; it returns every outcome.
4. `js async function show() { const value = await Promise.resolve("hi"); console.log(value); }`

5. b - an object with `Symbol.asyncIterator`.
6. False - `fetch` only rejects on network failure; check `res.ok`.
7. `await Promise.all([fetchA(), fetchB()])`

Chapter 10 - Modules and code quality

10.1 Summary

- ES modules are the standard: `export` and `import`, one file one module.
- Named exports and a single default export can be combined.
- Modules are singletons, cached per URL, strict by default.
- `import * as ns` imports everything as an object; `import()` is dynamic.
- Modules make dependencies explicit and avoid global variables.
- Circulation between modules is allowed but should be avoided.
- `console.log`, the debugger and unit tests are the tools of quality.
- Write small functions with one job and descriptive names.
- Comments explain why; the code explains what.
- A tiny test harness runs anywhere: `assert` or a one-file runner.

10.2 Concepts

Export and import

JSREF

```
1  // math.js
2  export const square = (n) => n * n;
3  export default function multiply(a, b) { return a * b; }
```

JSREF

```
1  // main.js
2  import multiply, { square } from "./math.js";
3  import * as math from "./math.js";
```

Form	Use
`export const x = 1`	named export
`export default x`	one default per module
`export { a, b }`	export existing bindings
`export * from "./m.js"`	re-export
`import def, { named } from "./m.js"`	both kinds
`const m = await import("./m.js")`	dynamic import

In the browser and in Node

HTML

```
<script type="module" src="main.js"></script>
```

shell

```
node main.mjs          # or "type": "module" in package.json
node --check main.mjs  # syntax check only
```

A tiny test

JavaScript

```
import assert from "node:assert/strict";

function test(name, fn) {
  try { fn(); console.log("ok -", name); }
  catch (error) { console.log("FAIL -", name, error.message); }
}
```

10.3 Examples

Example 1: named and default exports.

JavaScript

```
// MODULE EXAMPLE (syntax): two files, main.mjs and utils.mjs
// MODULE EXAMPLE (syntax): utils.mjs
export const greet = (name) => `Hello, ${name}`;
export const VERSION = "1.0.0";
export default function add(a, b) { return a + b; }
```

JavaScript

```
// MODULE EXAMPLE (syntax): main.mjs
import add, { greet, VERSION } from "./utils.mjs";
console.log(greet("Ada"), add(2, 3), VERSION);
```

output

```
Hello, Ada 5 1.0.0
```

Example 2: dynamic import.

JavaScript

```
async function load() {
  const math = await import("./math.mjs");   // syntax
  return math.VERSION;
}
```

Example 3: a passing test file.

JavaScript

```javascript
import assert from "node:assert/strict";

const sum = (...nums) => nums.reduce((a, b) => a + b, 0);

assert.equal(sum(1, 2, 3), 6);
assert.deepEqual([1, 2].concat([3]), [1, 2, 3]);
assert.throws(() => JSON.parse("{"));
console.log("all tests passed");
```

output

```
all tests passed
```

Example 4: a small test runner.

JavaScript

```javascript
const tests = [];
const test = (name, fn) => tests.push({ name, fn });

test("adds", () => { if (1 + 1 !== 2) throw new Error("bad"); });
test("strings", () => { if ("a" + "b" !== "ab") throw new Error("bad"); });

let failed = 0;
for (const { name, fn } of tests) {
  try { fn(); console.log("ok -", name); }
  catch (error) { failed++; console.log("FAIL -", name); }
}
console.log(failed ? failed + " failed" : "all " + tests.length + " passed");
```

output

```
ok - adds
ok - strings
all 2 passed
```

Example 5: useful comments.

JavaScript

```javascript
// Calculate the price including VAT.
// VAT is applied to the net price, not to the shipping cost.
function total(net, shipping) {
  const vat = net * 0.21;
  return net + vat + shipping;
}
console.log(total(100, 10));
```

output

```
131
```

10.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Scripts that set globals	collisions	ES modules

Mistake	Why it hurts	Best practice
One giant `utils.js`	unclear dependencies	focused modules
`export default` everywhere	inconsistent names	prefer named exports
Circular imports	confusing initialisation order	extract the shared part
Commenting every line	noise	comment the why
No tests	regressions slip in	a tiny runner, run often

- Keep a module's public surface small: export what is needed, not everything.
- `node --check` catches syntax errors before you run the file.

10.5 Practice exercises

1. Export a named function and import it in another module.
2. Which two commands validate and run a `.mjs` file?
3. Write a test that fails if `"a".toUpperCase()` is not `"A"`.
4. Why prefer named exports over default exports?
5. What does a circular import do to initialisation order?

Solutions

1. `// m.js` with `export const slug = (s) => s.toLowerCase();` and `// main.js` with `import { slug } from "./m.js";`
2. `node --check file.mjs` and `node file.mjs`.
3. `js import assert from "node:assert/strict"; assert.equal("a".toUpperCase(), "A");`
4. Because the name is explicit at the import site, refactoring tools and readers can follow it, and the same module can export several things without inventing names.
5. The module that is imported first may see `undefined` for the bindings of the module that is still initialising, which is why circular imports should be avoided or the shared code extracted into a third module.

10.6 Mini test

1. Multiple choice: which keyword starts an import? - a) `require` - b) `import` - c) `include` - d) `using`
2. True or false: a module may have several default exports.
3. Multiple choice: which module loading is dynamic? - a) `import x from "./m.js"` - b) `await import("./m.js")` - c) `import * as x` - d) `export`
4. Short code question: export a constant named `MAX`.
5. Multiple choice: which command only checks syntax? - a) `node file.mjs` - b) `node --check file.mjs` - c) `node -e "x"` - d) `npm test`
6. True or false: modules are strict by default.

7. Short answer: what should a comment explain?

Answer key and explanations

1. b - `import`.
2. False - exactly one default export per module.
3. b - `await import(...)` loads on demand and returns a promise.
4. `export const MAX = 10;`
5. b - `--check` parses without executing.
6. True - module code is always strict, so silent errors throw.
7. Why the code exists, not what it does line by line.

Chapter 11 - The DOM

11.1 Summary

- The DOM is a tree of nodes built from the HTML document.
- `querySelector` and `querySelectorAll` select with CSS selectors.
- Element properties (`textContent`, `className`) and attributes differ.
- `createElement`, `append`, `remove` build and change the tree.
- `dataset` reads `data-*` attributes as camelCase properties.
- `classList` adds, removes, toggles and checks classes.
- The DOM is slow to write in bulk: batch changes or use fragments.
- `innerHTML` parses strings: never use it with untrusted content.
- `textContent` is the safe alternative for text.
- Read the dev tools Elements panel to see the live tree.

11.2 Concepts

Selecting

JSREF

```
document.querySelector("main h1");     // the first match
document.querySelectorAll(".item");    // every match, a NodeList
document.getElementById("hero");       // by id
document.forms["signup"];              // a form by name
```

Reading and writing

Property	Meaning
`textContent`	the text of the element and its children (safe)

Property	Meaning
`innerHTML`	the HTML inside (parses strings, XSS risk)
`outerHTML`	the element including its own tag
`id`, `className`, `classList`	identity and classes
`attributes`, `getAttribute`, `setAttribute`	attributes
`dataset`	`data-*` values
`value`	for inputs, selects and textareas
`style`	inline styles (prefer classes)
`hidden`, `disabled`, `checked`	state properties

Creating and inserting

JSREF

```
const li = document.createElement("li");
li.textContent = "New";
list.append(li);            // also prepend, before, after, replaceWith
list.removeChild(li);
fragment.append(...items);  // one insertion, many nodes
```

Traversal

JSREF

```
el.parentElement; el.children; el.firstElementChild; el.lastElementChild;
el.nextElementSibling; el.previousElementSibling; el.closest("li");
```

11.3 Examples

Example 1: query and read.

JavaScript

```
// DOM EXAMPLE
const heading = document.querySelector("h1");
const items = document.querySelectorAll(".item");
console.log(heading.textContent, items.length);
```

Expected result: the heading text and the number of `.item` elements.

Example 2: build a list.

JavaScript

```
// DOM EXAMPLE
const list = document.querySelector("ul");
const fragment = document.createDocumentFragment();
for (const name of ["Ada", "Alan", "Grace"]) {
  const li = document.createElement("li");
  li.textContent = name;
  fragment.append(li);
}
list.append(fragment);
```

Expected result: three list items added with a single insertion.

Example 3: classes and data attributes.

JavaScript

```
// DOM EXAMPLE
const card = document.querySelector(".card");
card.classList.add("is-active");
card.classList.toggle("is-hidden");
console.log(card.dataset.userId, card.dataset.role, card.classList.contains("is-active"));
```

Expected result: the values of `data-user-id` and `data-role`, then `true`.

Example 4: attributes versus properties.

JavaScript

```
// DOM EXAMPLE
const link = document.querySelector("a");
console.log(link.href, link.getAttribute("href"));
link.setAttribute("target", "_blank");
link.rel = "noopener";
```

Expected result: `href` returns the resolved absolute URL while `getAttribute("href")` returns the text written in the HTML.

Example 5: why `innerHTML` is dangerous.

JavaScript

```
// DOM EXAMPLE
const userInput = '<img src=x onerror="alert(1)">';
element.textContent = userInput;    // safe: shows the text
// element.innerHTML = userInput;   // dangerous: parses and executes
```

Expected result: the first line renders the input as text; the commented line would execute the `onerror` handler.

11.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`innerHTML` with user content	XSS	`textContent`
Querying inside a loop	slow	query once, cache the result
Writing in a loop	layout thrashing	`DocumentFragment`, one insert

Mistake	Why it hurts	Best practice
Inline `style` from JS everywhere	hard to maintain	toggle classes
`document.write`	blocks and wipes the page	DOM methods
Storing data in classes	fragile	`dataset` or a `Map`

- Cache DOM references; do not re-query on every event.
- Let CSS own presentation: change classes or data attributes, not styles.

11.5 Practice exercises

1. Select the first `p` inside `main`.
2. Create a `button` with the text "Save" and append it to a form.
3. Read `data-id` from an element.
4. Why is `textContent` safer than `innerHTML`?
5. Toggle the class `open` on a dialog and report the new state.

Solutions

1. `document.querySelector("main p")`
2. `js const button = document.createElement("button"); button.textContent = "Save"; form.append(button);`
3. `element.dataset.id`
4. Because `textContent` never parses HTML, so user input cannot inject elements or execute scripts.
5. `js const nowOpen = dialog.classList.toggle("open"); console.log(nowOpen);`

11.6 Mini test

1. Multiple choice: which method returns the first match? - a) `querySelectorAll` - b) `querySelector` - c) `getElementsByClassName` - d) `getElementById` (also first)
2. True or false: `innerHTML` with untrusted text is an XSS risk.
3. Multiple choice: which collection holds `data-*` values? - a) `el.attributes` - b) `el.dataset` - c) `el.data` - d) `el.storage`
4. Short code question: add the class `active` to an element.
5. Multiple choice: what returns a `NodeList`? - a) `querySelector` - b) `querySelectorAll` - c) `createElement` - d) `getElementById`
6. True or false: `classList.toggle` returns the new state.
7. Short answer: why batch DOM writes?

Answer key and explanations

1. b - `querySelector` (as does `getElementById`).

2. True - the browser parses the string, which can run scripts.
3. b - `dataset`, camelCased from `data-*`.
4. `element.classList.add("active");`
5. b - `querySelectorAll` returns a `NodeList`.
6. True - it returns whether the class is now present.
7. Because each write can force layout and layout thrashing janks the page: one insertion of a `DocumentFragment` is far cheaper than many.

Chapter 12 - Events and forms

12.1 Summary

- Events flow through capture, target and bubble phases.
- `addEventListener(type, handler, options)` is the standard way to listen.
- The handler receives an `event` object with `target`, `currentTarget`, `type`.
- `preventDefault` stops the default action; `stopPropagation` stops bubbling.
- Event delegation listens on a parent and uses `closest()` to find the item.
- Forms fire `submit`; validate before sending, always again on the server.
- `input`, `change`, `focus`, `blur` cover most form interactions.
- `checkValidity()` and `setCustomValidity()` use the browser's validation.
- `AbortController` removes many listeners at once.
- Remove listeners on elements you delete to avoid leaks.

12.2 Concepts

Phases

output

```
window -> document -> html -> body -> ... -> target  (capture)
target                                          (target)
... -> body -> html -> document -> window  (bubble)
```

Listeners run in the bubble phase by default; pass `capture: true` to invert.

The event object

Property	Meaning
`type`	`"click"`, `"submit"`…
`target`	the element that received the event
`currentTarget`	the element the listener is attached to
`bubbles`, `cancelable`	what is allowed

Property	Meaning
`defaultPrevented`	whether `preventDefault` was called

Useful events

Category	Events
mouse	`click`, `dblclick`, `mouseover`, `mouseout`, `contextmenu`
keyboard	`keydown`, `keyup`, `keypress` (legacy)
form	`submit`, `input`, `change`, `focus`, `blur`, `invalid`
document	`DOMContentLoaded`, `scroll`, `resize`, `visibilitychange`
pointer	`pointerdown`, `pointermove`, `pointerup`

Forms

JSREF

```
const form = document.querySelector("form");
form.addEventListener("submit", (event) => {
  event.preventDefault();
  const data = new FormData(form);      // field name to value
  const object = Object.fromEntries(data);
});
```

12.3 Examples

Example 1: a click handler.

JavaScript

```
// DOM EXAMPLE
const button = document.querySelector("button");
button.addEventListener("click", (event) => {
  console.log(event.type, event.target === button, event.defaultPrevented);
});
```

Expected result: `click true false` in the console when the button is clicked.

Example 2: event delegation.

JavaScript

```
// DOM EXAMPLE
document.querySelector("ul").addEventListener("click", (event) => {
  const li = event.target.closest("li");
  if (!li) return;
  console.log(li.dataset.id, event.currentTarget.tagName);
});
```

Expected result: the `data-id` of the clicked row and `UL`, no matter how many rows exist or are added later.

Example 3: submit with `preventDefault`.

JavaScript

```
// DOM EXAMPLE
const form = document.querySelector("form");
form.addEventListener("submit", (event) => {
  event.preventDefault();
  const data = new FormData(form);
  console.log(data.get("name"), data.has("agree"));
});
```

Expected result: the field values logged instead of the page reloading.

Example 4: live validation.

JavaScript

```
// DOM EXAMPLE
const input = document.querySelector("#email");
input.addEventListener("input", () => {
  const valid = input.checkValidity();
  input.setAttribute("aria-invalid", String(!valid));
  console.log(input.value.length, valid);
});
```

Expected result: the length and validity on every keystroke.

Example 5: removing listeners with `AbortController`.

JavaScript

```
// DOM EXAMPLE
const controller = new AbortController();
const log = () => console.log("once");
button.addEventListener("click", log, { signal: controller.signal });
button.addEventListener("click", log, { signal: controller.signal });
controller.abort();          // both listeners removed
```

Expected result: clicking the button logs nothing after `abort()`.

12.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`onclick="..."` attributes	one handler, mixed concerns	`addEventListener`
Many listeners on many rows	slow, memory	delegate on the parent
Trusting client validation	trivially bypassed	always re-validate server-side
`stopPropagation` everywhere	breaks other features	only where needed
Listeners on removed nodes	memory leak	`AbortController` or `removeEventListener`
Ignoring keyboard users	inaccessible	real buttons, focus styles

- Use `preventDefault` to keep the page from reloading on submit.
- `passive: true` for `scroll`/`touchmove` listeners improves scrolling.

12.5 Practice exercises

1. Log a click on any element inside a list using delegation.
2. Prevent a form from reloading the page on submit.
3. Read all form fields as an object.
4. Why re-validate on the server?
5. Remove every listener attached to a button at once.

Solutions

1. `js list.addEventListener("click", (event) => { const li = event.target.closest("li"); if (li) console.log(li.textContent); });`
2. `js form.addEventListener("submit", (event) => event.preventDefault());`
3. `Object.fromEntries(new FormData(form))`
4. Because client-side checks are a convenience only: they can be disabled, edited or replayed from outside the browser.
5. Attach them with `{ signal: controller.signal }`, then `controller.abort()`.

12.6 Mini test

1. Multiple choice: which method attaches a listener? - a) `on("click", fn)` - b) `addEventListener("click", fn)` - c) `listen("click", fn)` - d) `bind("click", fn)`
2. True or false: events bubble from the target to the root by default.
3. Multiple choice: which object collects form fields? - a) `FormInfo` - b) `FormData` - c) `Fields` - d) `URLSearchParams` (only the query string)
4. Short code question: stop a form from submitting the page.
5. Multiple choice: which option makes a scroll listener passive? - a) `{ passive: true }` - b) `{ capture: true }` - c) `{ once: true }` - d) `{ signal: null }`
6. True or false: `event.target` is always the element with the listener.
7. Short answer: why use event delegation?

Answer key and explanations

1. b - `addEventListener`.
2. True - the bubble phase runs listeners on ancestors.
3. b - `FormData` (then `Object.fromEntries` for a plain object).
4. `event.preventDefault();`
5. a - `passive: true` tells the browser the handler will not prevent scrolling, so it can scroll smoothly.

6. False - `target` is where the event happened; `currentTarget` is the listener's element.
7. One listener handles every current and future child, which is faster and avoids attaching listeners to dynamically added elements.

Chapter 13 - Storage, fetch and modern extras

13.1 Summary

- `localStorage` persists strings per origin; `sessionStorage` per tab.
- Storage is synchronous and capped: keep small data only.
- `fetch` returns promises; check `res.ok` and parse with `res.json()`.
- `AbortController` cancels a request and its listeners.
- `structuredClone` deep-copies data, including Dates and Maps.
- `URL` and `URLSearchParams` build and parse URLs safely.
- `Intl` formats dates, numbers and currencies per locale.
- `Proxy` and `Reflect` intercept operations on objects.
- `BigInt` handles integers beyond `Number.MAX_SAFE_INTEGER`.
- `queueMicrotask` schedule work after the current task, before rendering.

13.2 Concepts

Storage

JSREF

```
localStorage.setItem("theme", "dark");
localStorage.getItem("theme");           // "dark"
JSON.parse(localStorage.getItem("user") || "null");
```

`fetch`

JSREF

```
const res = await fetch("/api/items", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "x" }),
});
if (!res.ok) throw new Error("HTTP " + res.status);
const data = await res.json();
```

`URL`

JavaScript

```
const url = new URL("/search", "https://example.com");
url.searchParams.set("q", "css grid");
url.toString();
```

output

```
https://example.com/search?q=css+grid
```

13.3 Examples

Example 1: `structuredClone`.

JavaScript

```
const source = { user: { name: "Ada", tags: ["a"] } };
const copy = structuredClone(source);
copy.user.tags.push("b");
console.log(source.user.tags.join(","), copy.user.tags.join(","));
```

output

```
a a,b
```

Example 2: `BigInt`.

JavaScript

```
console.log(typeof 10n, 10n + 5n, Number(10n));
console.log(2n ** 80n > Number.MAX_SAFE_INTEGER);
console.log((12345678901234567890n).toString().length);
```

output

```
bigint 15n 10
true
20
```

Example 3: `Symbol` and `Proxy`.

JavaScript

```
const id = Symbol("id");
console.log(id.description, id === Symbol("id"));

const logged = new Proxy({}, {
  get: (target, key) => `get:${String(key)}`,
  set: (target, key, value) => { target[key] = value; return true; },
});
console.log(logged.anything);
logged.x = 1;
console.log(logged.x);
```

output

```
id false
get:anything
1
```

Example 4: regular expressions.

JavaScript

```
console.log("2026-10-10".replace(/(\d{4})-(\d{2})-(\d{2})/, "$3/$2/$1"));
console.log("a1b22c333".match(/\d+/g).join(","));
console.log(/\d{4}/.test("ab2026"), /^\d+$/.test("42"));
console.log("Ada Lovelace".split(" ").map((w) => w[0]).join("."));
```

output

```
10/10/2026
1,22,333
true true
A.L.
```

Example 5: `URL` and `URLSearchParams`.

JavaScript

```
const url = new URL("/search", "https://example.com");
url.searchParams.set("q", "css grid");
url.searchParams.append("page", "2");
console.log(url.toString());
console.log(url.searchParams.get("q"), [...url.searchParams.keys()].join(","));
```

output

```
https://example.com/search?q=css+grid&page=2
css grid q,page
```

Example 6: `Intl` formatting.

JavaScript

```
const money = new Intl.NumberFormat("en-GB", { style: "currency", currency: "EUR" });
const date = new Intl.DateTimeFormat("en-GB", { dateStyle: "medium" });
console.log(money.format(1234.5));
console.log(date.format(new Date(2026, 9, 10)));
console.log(new Intl.NumberFormat("en-GB").format(0.256));
```

output

```
EUR1,234.50
10 Oct 2026
0.256
```

Example 7: storage helper.

JavaScript

```
// DOM EXAMPLE
const store = {
  get(key, fallback) {
    try { return JSON.parse(localStorage.getItem(key)) ?? fallback; }
    catch { return fallback; }
  },
  set(key, value) { localStorage.setItem(key, JSON.stringify(value)); },
};
store.set("theme", "dark");
console.log(store.get("theme"), store.get("missing", "light"));
```

Expected result: `dark light` in the browser console.

Example 8: fetch with a timeout.

JavaScript

```javascript
// DOM EXAMPLE
async function getJson(url, timeout = 5000) {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), timeout);
  try {
    const res = await fetch(url, { signal: controller.signal });
    if (!res.ok) throw new Error("HTTP " + res.status);
    return await res.json();
  } finally {
    clearTimeout(timer);
  }
}
```

13.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Storing objects directly	stores strings only	`JSON.stringify` on write, parse on read
Ignoring `res.ok`	errors look like data	check the status
No timeout	hanging UI	`AbortController`
Huge objects in `localStorage`	quota errors (~5MB)	keep it to preferences and tokens
`new Date(string)`	locale parsing quirks	`Intl` for formatting, ISO for parsing
Manual regex for URLs	bugs	`URL`/`URLSearchParams`

- Never store secrets in `localStorage`: any script on the page can read it.
- Use `Intl` instead of hand-rolled formatting.

13.5 Practice exercises

1. Deep copy an object containing an array.
2. Format `1234.5` as euros in British English.
3. Cancel a `fetch` after three seconds.
4. Read and write a JSON value in `localStorage` safely.
5. Build the URL `https://x.test/s?q=a%20b`.

Solutions

1. `structuredClone(source)`
2. `new Intl.NumberFormat("en-GB", { style: "currency", currency: "EUR" }).format(1234.5)`
3. `js const controller = new AbortController(); setTimeout(() => controller.abort(), 3000); fetch(url, { signal: controller.signal });`

4. `js const read = (k, d) => { try { return JSON.parse(localStorage.getItem(k)) ?? d; } catch { return d; } }; const write = (k, v) => localStorage.setItem(k, JSON.stringify(v));`
5. `js const url = new URL("https://x.test/s"); url.searchParams.set("q", "a b");`

13.6 Mini test

1. Multiple choice: which method deep-copies plain data? - a) `{ ...o }` - b) `structuredClone(o)` - c) `Object.assign` - d) `JSON.parse(JSON.stringify(o))` for data without Dates or Maps
2. True or false: `fetch` rejects on a 404 response.
3. Multiple choice: which API cancels a request? - a) `CancelToken` - b) `AbortController` - c) `AbortSignal` alone - d) `fetch.abort()`
4. Short code question: read a JSON value from `localStorage` with a default.
5. Multiple choice: which constructor formats currency? - a) `Intl.NumberFormat` - b) `Intl.Currency` - c) `Number.toCurrency` - d) `money()`
6. True or false: `localStorage` survives closing the browser.
7. Short answer: why not store tokens in `localStorage`?

Answer key and explanations

1. b - (d) also works but loses Dates and Maps, and `structuredClone` is faster and safer.
2. False - only network failures reject; a 404 resolves with `res.ok === false`.
3. b - `AbortController` (via its `signal`).
4. `js JSON.parse(localStorage.getItem("key") || "null") ?? defaultValue`
5. a - `Intl.NumberFormat` with `style: "currency"`.
6. True - `localStorage` persists per origin; `sessionStorage` does not survive the tab.
7. Because any script running on the page (including a third-party one) can read it, so a successful XSS also leaks the session.

Cheat sheet

14.1 Running and syntax

shell

```shell
node file.mjs          # run
node --check file.mjs  # syntax only
```

JavaScript

```javascript
const x = 1;           // constant binding
let y = 2;             // reassignable
"use strict";          // default in modules
// comment             /* block comment */
```

14.2 Types and checks

JSREF

```
typeof x              // "string" "number" "bigint" "boolean"
                      // "undefined" "symbol" "function" "object"
Array.isArray(x)      // arrays
Number.isNaN(x)       // the safe NaN test
Number.isInteger(x)   // whole numbers
Number.isFinite(x)    // not NaN and not Infinity
x === null            // the only reliable null test
```

14.3 Numbers

JSREF

```
Number.parseInt("12px", 10)   // 12
Number.parseFloat("1.5kg")   // 1.5
(1.005).toFixed(2)           // "1.00"
(255).toString(16)           // "ff"
Math.max(...arr) Math.min(...arr)
Math.round(2.5) Math.floor(-2.5) Math.ceil(2.1)
Math.abs(-5) Math.sqrt(16) 2 ** 10
```

14.4 Strings

JSREF

```
`Hello, ${name}`           // template literal
s.trim() s.toUpperCase() s.toLowerCase()
s.slice(0, 5) s.slice(-3)
s.split(", ") s.replaceAll("a", "b")
s.includes("x") s.startsWith("a") s.endsWith("z")
s.indexOf("x") s.padStart(3, "0") s.repeat(2)
s.match(/\d+/g) s.replace(/(\w+)@/, "$1 at ")
```

14.5 Arrays

JSREF

```
a.map(f) a.filter(f) a.reduce(f, init)
a.find(f) a.findIndex(f) a.some(f) a.every(f)
a.includes(v) a.indexOf(v) a.at(-1) a.length
a.slice(1, 3) a.concat(b) [...a, ...b]
a.flat(Infinity) [...new Set(a)]
a.sort(cmp) a.reverse() a.push(v) a.pop()
a.splice(i, n) a.forEach(f)
```

JavaScript

```
// sort numbers, not lexicographically
[10, 9, 80].sort((a, b) => a - b)
```

14.6 Objects and classes

JSREF

```
{ key: value, method() {} }
Object.keys(o) Object.values(o) Object.entries(o)
Object.freeze(o) Object.fromEntries(entries)
{ ...o }                  // shallow copy
structuredClone(o)        // deep copy
const { a, b = 1 } = o
class A extends B { #priv = 1; static s = 2; get x() {} }
new A() instanceof A
```

14.7 Map, Set, JSON

JSREF

```
new Map([["a", 1]])   new Set([1, 1, 2])
m.set(k, v) m.get(k) m.has(k) m.delete(k) m.size
[...m.keys()] [...m.values()] [...m.entries()]
JSON.stringify(v)      JSON.parse(text)
JSON.stringify(v, null, 2)   // pretty
```

14.8 Functions

JSREF

```
function f(a, b = 1, ...rest) {}
const arrow = (a) => a * 2;
f.call(obj, ...) f.apply(obj, []) f.bind(obj, ...)
const curry = (a) => (b) => a + b;
const add = (a) => (b) => a + b;
```

14.9 Scope and closures

JavaScript

```
let perIteration = 1;   // block scoped
// var functionScoped; // avoid
```

14.10 Errors

JSREF

```
try { risky(); } catch (error) {
  console.error(error.name, error.message);
} finally { cleanup(); }

throw new TypeError("...");
class MyError extends Error { constructor(m) { super(m); this.name = "MyError"; } }
throw new Error("wrapped", { cause: original });
```

14.11 Async

JSREF

```
async function f() {
  const res = await fetch(url);
  if (!res.ok) throw new Error("HTTP " + res.status);
  return res.json();
}
Promise.all(ps) Promise.allSettled(ps) Promise.any(ps) Promise.race(ps)
for await (const x of iterable) {}
```

output

```
output order: sync code, microtasks (promises), macrotasks (timers)
```

14.12 DOM and events

JSREF

```
document.querySelector(sel)      document.querySelectorAll(sel)
el.textContent                  // safe text
el.innerHTML                    // XSS risk with user data
el.classList.add/remove/toggle/contains
el.dataset.id                   // data-id
document.createElement(tag)     el.append(node)
el.addEventListener("click", fn, { signal, passive, once, capture })
event.target event.currentTarget event.preventDefault()
event.target.closest("li")      // delegation
new FormData(form) Object.fromEntries(new FormData(form))
new AbortController()           // cancel fetch and listeners
```

14.13 Storage, URL, Intl

JSREF

```
localStorage.getItem/setItem    // strings only, JSON encode
new URL(path, base).searchParams.set("q", "a b")
new Intl.NumberFormat("en-GB", { style: "currency", currency: "EUR" })
new Intl.DateTimeFormat("en-GB", { dateStyle: "medium" })
```

14.14 The traps to remember

Trap	Remember
`0.1 + 0.2 !== 0.3`	floating point
`"5" + 3 === "53"`	`+` concatenates
`"5" - 3 === 2`	`-` converts
`typeof null === "object"`	historical quirk
`NaN === NaN` is false	use `Number.isNaN`
`[10, 9].sort()` sorts as text	pass a comparator
`{ ...o }` is one level deep	`structuredClone` for nested

Trap	Remember
arrow methods lose `this`	use regular methods
`var` in a loop shares one binding	use `let`
`fetch` resolves on HTTP errors	check `res.ok`
`forEach(async ...)` does not wait	`for...of` or `Promise.all`
`innerHTML` parses user input	use `textContent`

Final test

1. Multiple choice: what does `typeof null` return? - a) `null` - b) `object` - c) `undefined` - d) `boolean`
2. True or false: `const obj = {}` prevents adding new properties.
3. Multiple choice: what is `"5" - 2`? - a) `"52"` - b) `3` - c) `NaN` - d) `"3"`
4. Short code question: declare a counter you will reassign.
5. Multiple choice: which method returns a new array? - a) `sort` - b) `push` - c) `map` - d) `splice`
6. Multiple choice: what does `[10, 9].sort()` produce? - a) `[9, 10]` - b) `[10, 9]` - c) `[10, 9]` reversed per engine - d) an error
7. Short code question: sort `[10, 9, 80]` numerically.
8. Multiple answer: which are primitive types? (`string`, `object`, `number`, `symbol`, `function`, `bigint`)
9. Multiple choice: which function returns `this` bound permanently? - a) `call` - b) `apply` - c) `bind` - d) `return`
10. Short code question: a closure that counts up.
11. Multiple choice: what does reading `null.x` throw? - a) `RangeError` - b) `TypeError` - c) `SyntaxError` - d) `ReferenceError`
12. True or false: `finally` runs even when the `try` block returns.
13. Short code question: await a promise and catch its error.
14. Multiple choice: which runs first, a `setTimeout(0)` or a resolved promise's `then`? - a) the timer - b) the promise - c) in source order - d) neither
15. True or false: `fetch` rejects when the server returns 404.
16. Multiple choice: which structure stores unique values? - a) `Map` - b) `Set` - c) `Object` - d) `WeakMap`
17. Short code question: convert `[1, 1, 2]` to unique values.
18. Multiple choice: what does `JSON.stringify(undefined)` return? - a) `"undefined"` - b) `undefined` - c) `null` - d) it throws
19. Multiple choice: which selector finds an element by id? - a) `#id` - b) `.id` - c) `[id]` - d) `get("id")`
20. Short code question: attach a click listener to a button.
21. Multiple choice: which is the safe way to insert user text? - a) `innerHTML` - b) `textContent` - c) `document.write` - d) `insertAdjacentHTML`

22. True or false: event delegation works for elements added later.
23. Short code question: prevent a form submission from reloading.
24. Multiple choice: which deep-copies an object? - a) `{ ...o }` - b) `Object.assign({}, o)` - c) `structuredClone(o)` - d) `o.slice()`
25. Multiple choice: what does `10n + 5n` return? - a) `15` - b) `15n` - c) `100n` - d) a TypeError
26. Short code question: multiply 2 by 80 as a BigInt and compare with the largest safe integer.
27. Multiple choice: which method formats currency? - a) `Intl.NumberFormat` - b) `Number.toCurrency` - c) `formatMoney` - d) `Intl.Currency`
28. True or false: `localStorage` stores strings only.
29. Short code question: read a JSON value from `localStorage` with a default.
30. Multiple choice: which command checks syntax without running? - a) `node file.mjs` - b) `node --check file.mjs` - c) `node -c file.mjs` - d) `npm run check`

15.1 Final test answers

1. b - `"object"`, a long-standing quirk; test `x === null` instead.
2. False - `const` protects the binding, not the object, so properties can still be added (`Object.freeze` prevents that).
3. b - `3`, because `-` converts both operands to numbers.
4. `let counter = 0;`
5. c - `map` returns a new array; the others mutate in place.
6. b - `sort` compares strings, and `"10" < "9"`, so the order stays `[10, 9]`.
7. `[10, 9, 80].sort((a, b) => a - b)`
8. `string`, `number`, `symbol`, `bigint` (`object` and `function` are not primitives).
9. c - `bind`.
10. `js function makeCounter() { let count = 0; return () => ++count; }`
11. b - `TypeError`.
12. True - `finally` also runs when the `try` block returns (and it overrides the return if it returns itself).
13. `js async function run() { try { return await fn(); } catch (error) { console.error(error.message); } }`
14. b - promise reactions (microtasks) drain before timer callbacks (macrotasks).
15. False - `fetch` only rejects on network errors; a 404 resolves with `res.ok === false`.
16. b - `Set`.
17. `[...new Set([1, 1, 2])]`
18. b - `undefined`, because `undefined` has no JSON representation.
19. a - `#id`.
20. `button.addEventListener("click", () => console.log("clicked"));`
21. b - `textContent` never parses HTML, so user input cannot execute.

22. True - the listener is on the ancestor, so newly added children are covered without extra listeners.
23. `form.addEventListener("submit", (event) => event.preventDefault());`
24. c - `structuredClone` is recursive.
25. b - `15n`, a BigInt.
26. `js console.log(2n ** 80n > Number.MAX_SAFE_INTEGER);`
27. a - `Intl.NumberFormat` with `style: "currency"`.
28. True - values are converted to strings, so objects need `JSON.stringify`.
29. `js JSON.parse(localStorage.getItem("key") || "null") ?? defaultValue`
30. b - `node --check file.mjs`.

Sources

Every URL below was consulted while planning and writing this volume. No text, images or code blocks were copied from these pages; all explanations, examples, exercises, tests and answers were written from scratch.

- https://javascript.info/
- https://javascript.info/intro
- https://javascript.info/first-steps
- https://javascript.info/structure
- https://javascript.info/variables
- https://javascript.info/types
- https://javascript.info/operators
- https://javascript.info/comparison
- https://javascript.info/ifelse
- https://javascript.info/logical-operators
- https://javascript.info/while-for
- https://javascript.info/switch
- https://javascript.info/function-basics
- https://javascript.info/function-expressions
- https://javascript.info/arrow-functions-basics
- https://javascript.info/object-basics
- https://javascript.info/object
- https://javascript.info/object-methods
- https://javascript.info/constructor-new
- https://javascript.info/classes
- https://javascript.info/class-inheritance
- https://javascript.info/property-accessors

- https://javascript.info/data-types
- https://javascript.info/number
- https://javascript.info/string
- https://javascript.info/array
- https://javascript.info/array-methods
- https://javascript.info/iterable
- https://javascript.info/map-set
- https://javascript.info/destructuring-assignment
- https://javascript.info/date
- https://javascript.info/json
- https://javascript.info/advanced-functions
- https://javascript.info/closure
- https://javascript.info/rest-parameters-spread
- https://javascript.info/var
- https://javascript.info/bind
- https://javascript.info/currying-partials
- https://javascript.info/error-handling
- https://javascript.info/try-catch
- https://javascript.info/custom-errors
- https://javascript.info/async
- https://javascript.info/promise-basics
- https://javascript.info/promise-api
- https://javascript.info/async-await
- https://javascript.info/event-loop
- https://javascript.info/generators
- https://javascript.info/modules-intro
- https://javascript.info/import-export
- https://javascript.info/modules-dynamic-imports
- https://javascript.info/document
- https://javascript.info/searching-elements-dom
- https://javascript.info/basic-dom-node-properties
- https://javascript.info/modifying-document
- https://javascript.info/styles-and-classes
- https://javascript.info/dom-attributes-and-properties
- https://javascript.info/events
- https://javascript.info/introduction-browser-events
- https://javascript.info/bubbling-and-capturing
- https://javascript.info/event-delegation

- https://javascript.info/forms-controls
- https://javascript.info/forms-submit
- https://javascript.info/loading
- https://javascript.info/script-async-defer
- https://javascript.info/network
- https://javascript.info/fetch
- https://javascript.info/fetch-abort
- https://javascript.info/data-storage
- https://javascript.info/localstorage
- https://javascript.info/proxy
- https://javascript.info/bigint
- https://javascript.info/regular-expressions
- https://javascript.info/currying-partials
- https://developer.mozilla.org/en-US/docs/Web/JavaScript - used to check language semantics, built-in objects and output formatting
- https://nodejs.org/api/cli.html - `node --check` syntax validation
- https://nodejs.org/api/assert.html - the `assert` module used in the tests